

Emergent Complexity Initial Assignment

Aryan Joshi

September 26, 2026

1 Overview

For this assignment, I had to implement a website that simulates modifications of “Conway’s Game of Life” with different birth/survival counts. The website supports 2D and 3D game rules and randomly samples 100 rules for the user to investigate. The website can be viewed [here](#), and the code is on [GitHub](#).

1.1 Life Simulation

The classes `life.hpp` and `life3D.hpp`, both of which extend a `lifeGame` interface, implement the rules of the simulation. Both of these classes store a boolean array of on/off cells and the ability to update the array according to the current rule. To avoid edges creating unintended edge cases, I chose to “wrap around” the grids so cells on the bottom row can affect the top and vice versa.

1.2 Analysis

The class `analysis.hpp` looks at a single `lifeGame` and attempts to classify it into one of 4 categories

- **Stable**, meaning this rule settles into an unchanging pattern
- **Oscillating**, meaning this rule settles into a repeating pattern of some kind
- **Chaotic**, meaning this rule settles into no structure whatsoever
- **Complex & Organized**, meaning this rule doesn’t repeat predictably, but has interesting “lifelike” structures of some kind

To classify rules, I first ran them for a long time and checked if any board state matched a previous one (via hashing). If the period was 1, I classified the rule as **Stable**, otherwise it was **Oscillating**. To decide between the last two categories, I compute the Shannon Entropy of every 5×5 or $5 \times 5 \times 5$ subgrid of cells, and decided every rule with a high enough entropy was **Chaotic**, otherwise it was **Complex & Organized**.

Use of Entropy I chose to use Shannon entropy because, subjectively, “organized” rules should have a small number of repeating structures, such as gliders. On the other hand, “unorganized” rules should exhibit lots of random patterns. Because of this, I thought that entropy was a good heuristic to decide between the two, similar to how languages tend to have much lower entropy than random text.

Multiple starts One random start isn’t enough to judge a rule, so every rule is run from three random soups (different starting densities) and gets whichever category wins the majority vote. Ties go to the more interesting category.

1.3 Website & AI Usage

Unlike the sections above, the website was entirely AI Generated. It includes:

- A 2D and 3D game of life viewer that allows the user to randomize or clear the grid, adjust the simulation speed as the simulation continues, and adjust Birth/Survival counts in real-time
- An “Analysis” section at the bottom of the website that randomly samples many possible rules and uses my heuristics to classify them into one of many categories.

The website doesn’t reimplement anything. My C++ is compiled to WebAssembly, so the website runs the exact same simulation and classifier as my code. Newly born or drawn cells show up white and fade to blue as they age, which makes moving objects easy to spot. Clicking a rule in the Analysis section replays that exact run (same grid, density and random seed) in the viewer, so anything in this report can be reproduced on the site. With seed 2026 and the default settings, the Analysis section gives the same 100 rules and results as my command line program.

I also used AI (Claude Code) to write the scripts behind the experiments in this report ([experiments/](#) in the repo). They use my engine and classifier unchanged, and it helped me draft this report.

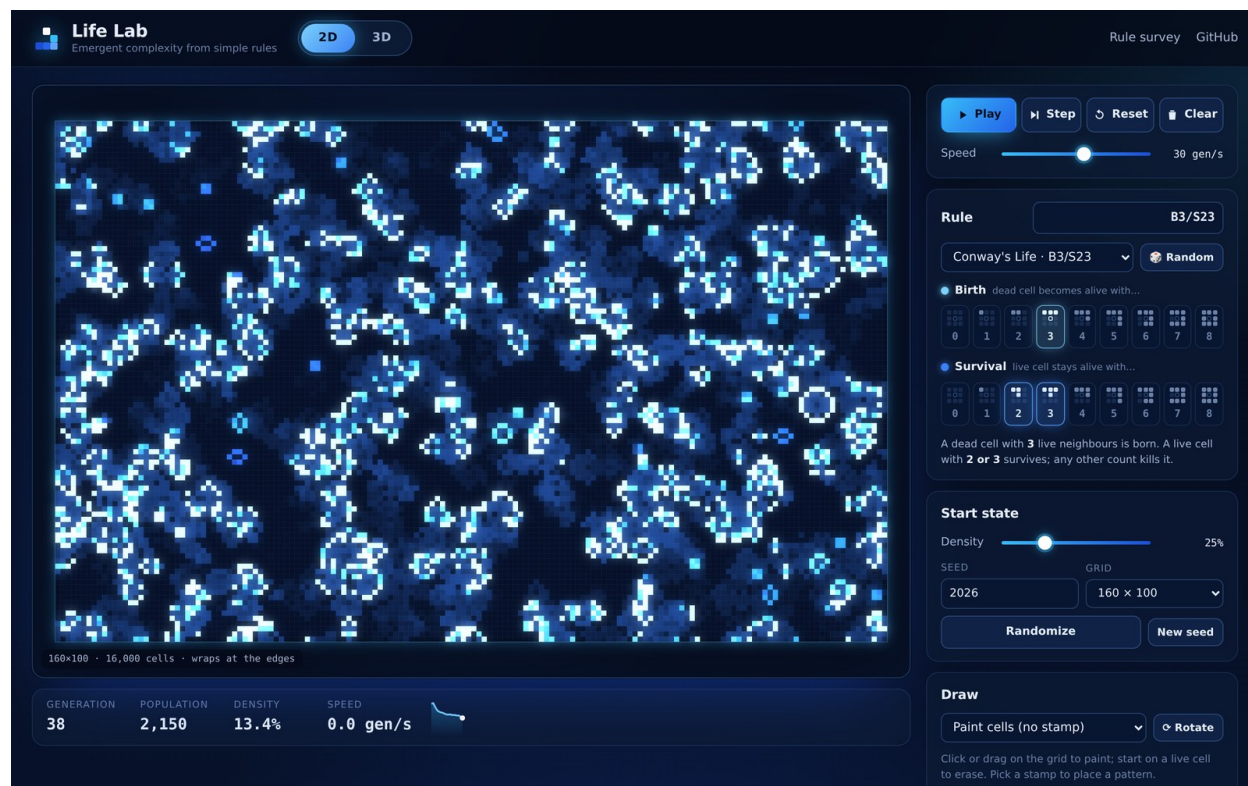


Figure 1: The website running Conway’s Life. White cells were just born, darker blue cells are older.

2 Exploring the Game of Life

I ran Conway’s Life (B3/S23) on a 128×128 grid from random soups at 12 starting densities between 5% and 80%, with 12 random seeds each (144 runs), until the board repeated or 6000 generations passed.

Every run went through the same three stages (Figure 2). First, most cells die in the first few generations from over- or under-crowding. Then there is a long messy phase of blobs growing, colliding and burning out, with gliders flying off and hitting things. Finally everything settles into “ash”: still lifes and blinkers. 136 of the 144 runs ended in a period 2 cycle.

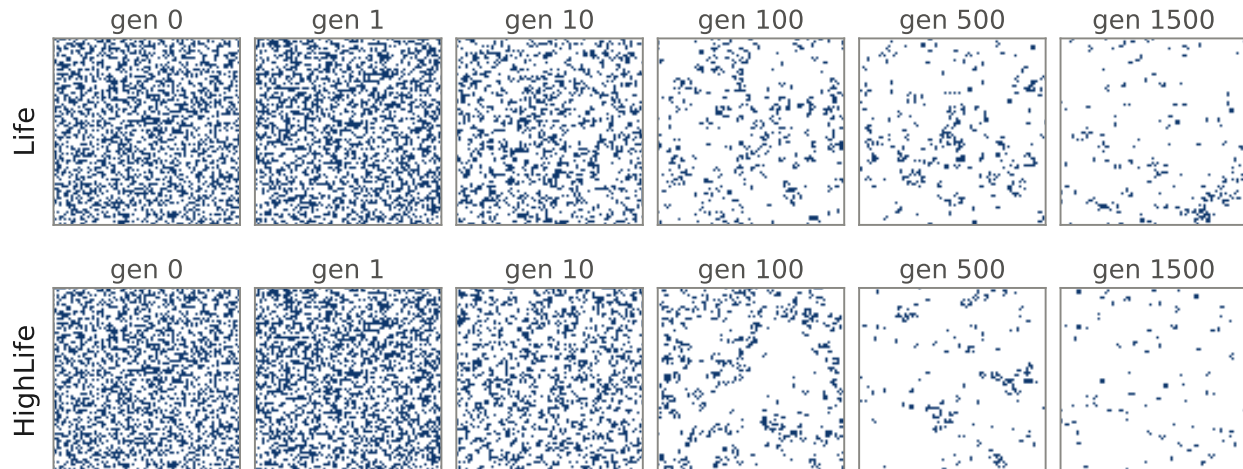


Figure 2: The same random soup (35% density) under Life (top) and HighLife (bottom).

Observations

- The ash is almost always the same density. For any starting density from 10% to 70%, the final board is $2.85\% \pm 0.32\%$ alive, even though settling takes anywhere from about 1200 to 2600 generations (median). Very sparse (5%) and very dense (80%) soups die almost immediately instead.
- Blinkers (34%), blocks (32%) and beehives (19%) make up 85% of everything that's left over. After those come loaves, boats, ponds, tubs, ships, toads, beacons and barges.
- Gliders are everywhere in the messy phase. Since my grid wraps around, a glider can't escape. Three runs ended with a period of 512, which is exactly the time a glider takes to travel around the 128-wide grid once.

3 Editable Rules and HighLife

On the website, the birth and survival counts are two rows of buttons numbered 0 to 8. Each button shows a small 3×3 picture with that many neighbours lit, so it's easy to see what each count means. You can also type a rule like B36/S23 or pick a preset. Clicking the “6” in the birth row turns Life into HighLife.

I repeated the soup experiment with HighLife (B36/S23). It looks almost identical to Life, but there are a few differences:

- The ash is thinner (1.97% alive vs. 2.85%), with more blocks (42% of objects) and far fewer blinkers (15%).
- Dense soups survive. At 80% density, Life dies within about 12 generations, but HighLife keeps going for over 1000, since birth on 6 neighbours lets cells appear inside crowded areas.
- One run ended in a period 14 cycle, the only period in HighLife other than 1 or 2.

The Replicator HighLife is known for its replicator, a 12 cell pattern that copies itself. Every 12 generations it becomes a row of copies, and when two copies land in the same spot they cancel out. The number of copies follows a Sierpinski triangle pattern: at generation $12k$ the population is exactly $12 \cdot 2^{(\text{number of 1s in } k \text{ in binary})}$. I checked this for $k = 0$ to 24 and it held every time.

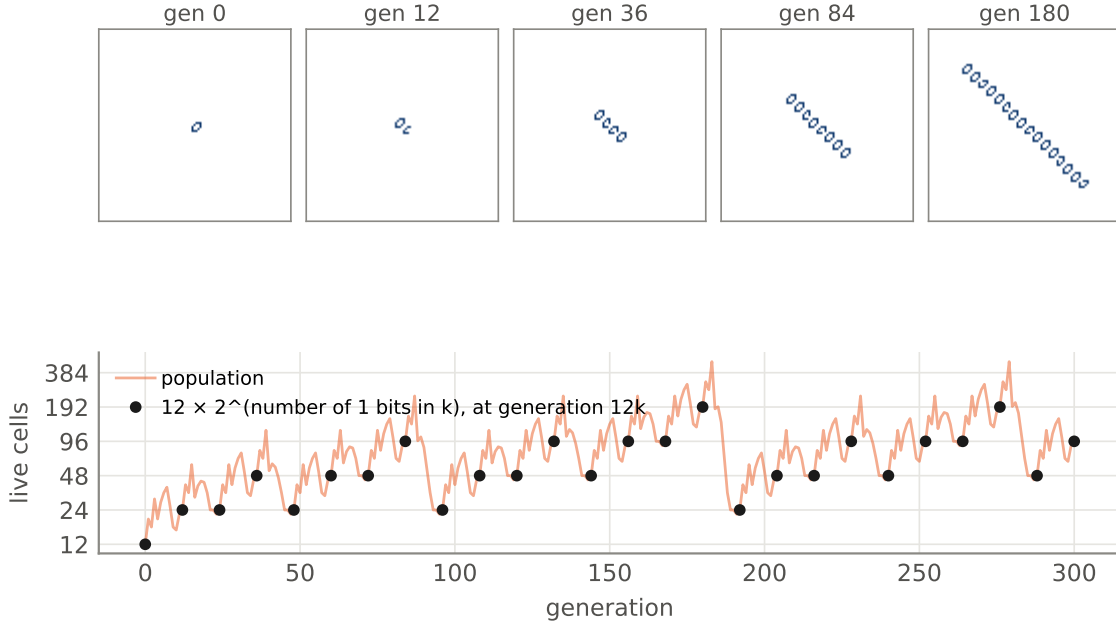


Figure 3: The HighLife replicator after 1, 2, 4, 8 and 16 copies, and its population compared to the formula (black dots).

Kind	HighLife patterns I found
Still lifes	block, beehive, loaf, boat, tub, pond, barge, long boat
Oscillators	blinker, toad, beacon (period 2), one period 14
Moving	glider (the same glider as Life)
Replicators	the 12 cell replicator

Table 1: Catalogue of HighLife patterns.

4 100 Random Rules

I sampled 100 random rules out of the 2^{18} possible ones and classified each on a 64×64 grid for up to 500 generations, starting from densities of 20%, 35% and 50%. To make sure 100 was enough, I also ran a second batch of 1000 rules.

Category	100 rules	1000 rules
Stable	9	66
Oscillating	19	173
Chaotic	71	757
Complex & Organized	1	4

Table 2: How the random rules were classified.

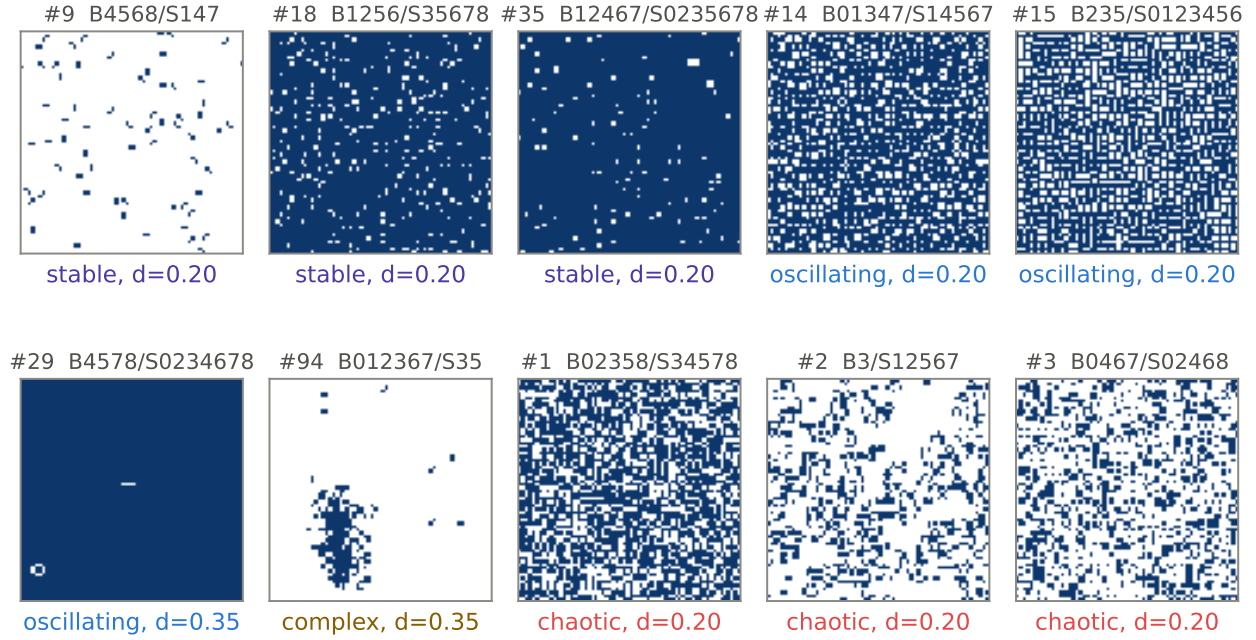


Figure 4: Final boards for some of the 100 sampled rules.

Observations

- Most random rules are chaotic and just fill the board with noise.
- The best predictor of chaos is the smallest birth count. If a dead cell can be born with 0, 1 or 2 neighbours, the rule was chaotic about 80% of the time, since those births happen almost everywhere. Out of the rules whose smallest birth count was 3 or more (like Life), only 39% were chaotic (Figure 5). Only 1 in 8 random rules avoids births on 0, 1 and 2, which is why chaos dominates.
- Simply counting how many birth/survival numbers a rule has (Langton's λ) doesn't separate the categories at all.
- Many oscillating rules have birth on 0 but not survival on 8, so the whole board flips on and off every generation.
- 9 of the 100 rules got different categories from different starting densities, so running a rule once really isn't enough.

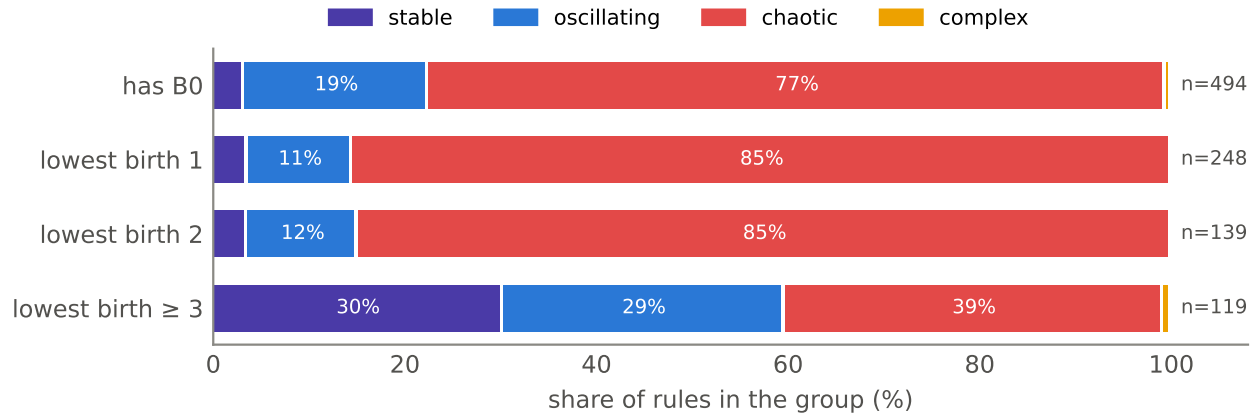


Figure 5: 1000 random rules, grouped by their smallest birth count.

4.1 A Closer Look: B012367/S35

The only rule my classifier called Complex & Organized was B012367/S35. When I watched it on the website, it didn't look organized at all: the whole board flashes between mostly empty and mostly full every generation (Figure 6). Its entropy is low because of large uniform areas, not because of gliders or other structures.

It turns out that a rule with birth on 0 but not survival on 8 can be rewritten exactly: if you invert every other frame, it becomes two normal rules taking turns (B458/S0124678 and B35/S125678). I checked this over 4000 generations and every single one matched. So this rule is really an ordinary rule seen through a flickering lens, and 3 of the 4 “complex” rules in the 1000 rule batch were the same kind. This showed me my entropy heuristic can be fooled.

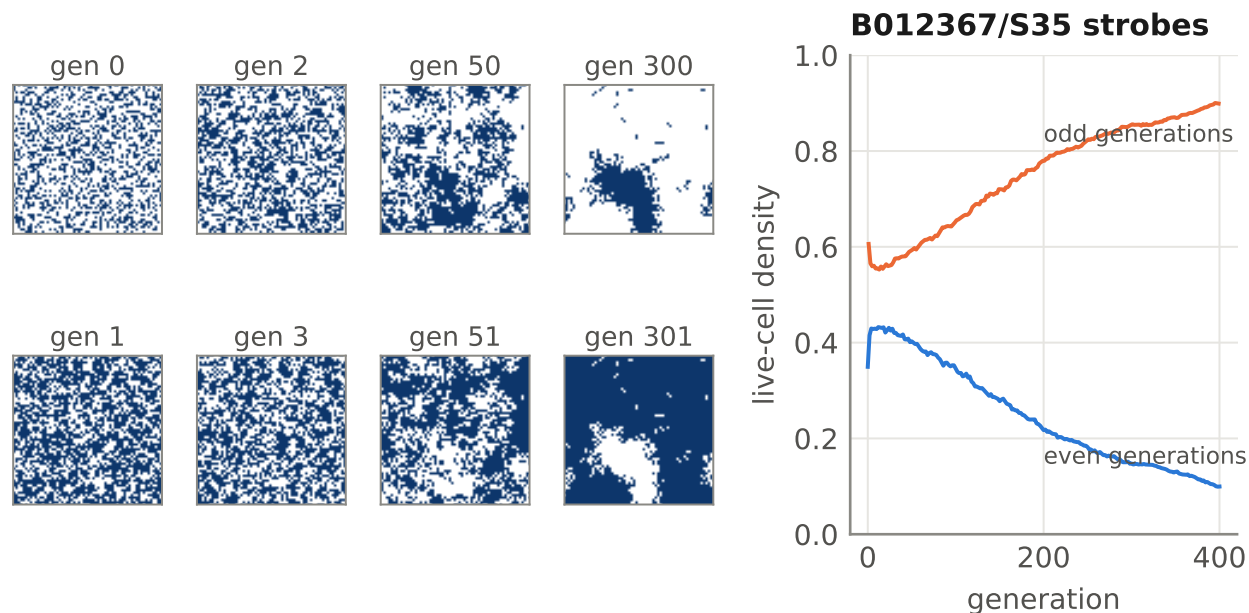


Figure 6: B012367/S35: even generations (top) and the odd generations right after (bottom) are nearly inverted.

Searching for more interesting rules If I were to search for better rules, I would:

- Only sample rules without birth on 0, 1 or 2, since that's where Life-like rules are.
- Convert flickering rules into their two normal rules before classifying them.
- Look for gliders directly (small shapes that reappear shifted a few cells later), since entropy doesn't notice them.
- Mutate rules I already know are interesting, like flipping one number in Life or HighLife.

5 3D Cellular Automata

For the open-ended part I picked Option A. `life3D.hpp` works exactly like the 2D version but with a 3D wrapped grid, so each cell has 26 neighbours and birth/survival counts go from 0 to 26. My classifier didn't need any changes, since the entropy just uses $5 \times 5 \times 5$ cubes instead of 5×5 squares.

On the website, the 3D grid is shown as cubes you can rotate and zoom around. To edit, you click a cube's face to stack a new cube on it, or click a glowing “layer” plane to place a cube on that layer. There is also an option to hide everything above the layer to see inside.

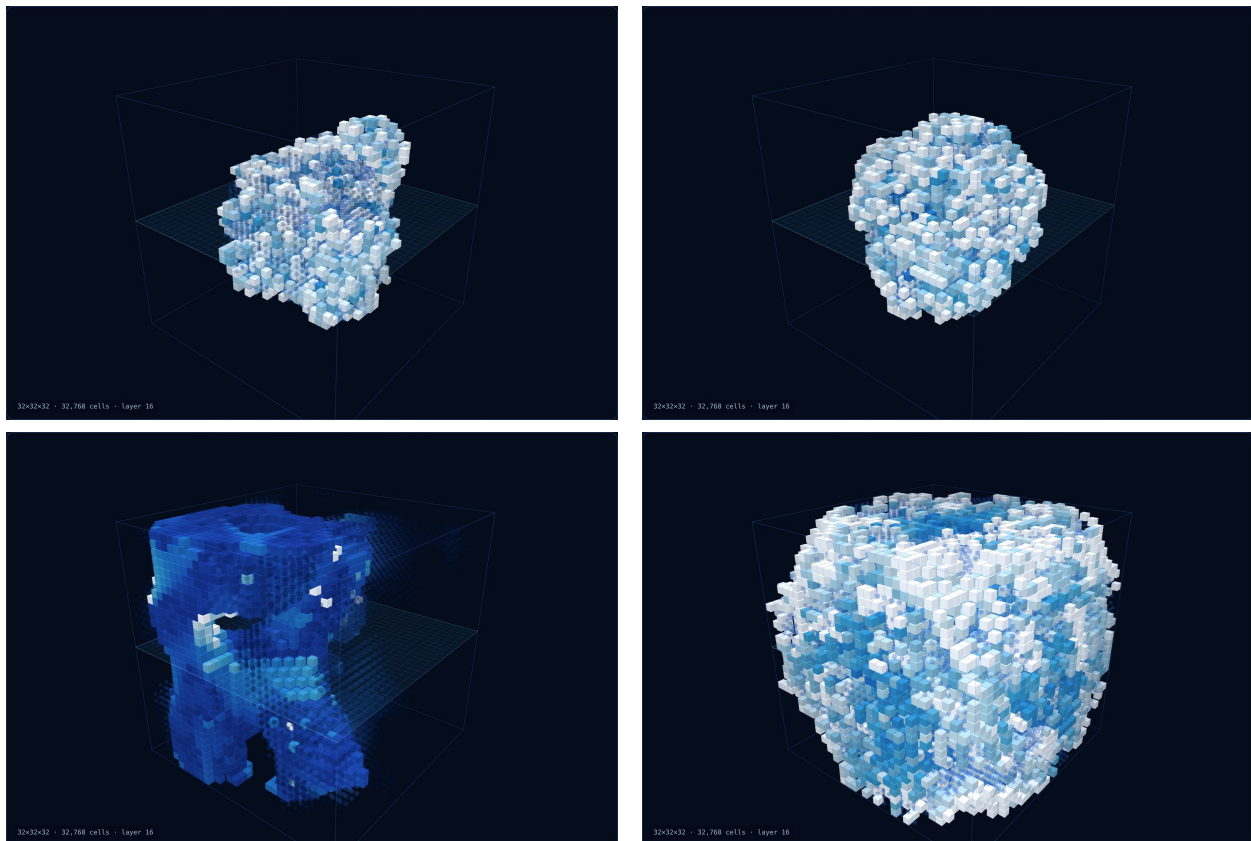


Figure 7: 3D rules on a 32^3 grid. Top: B6/S5678 growing like an amoeba, and B6/S45678 freezing into a ball. Bottom: “Clouds” (B13,14,17–19/S13–26) and B567/S5–11 growing like coral.

How big can the grid be? Every generation has to look at 26 neighbours for every cell, so 3D is much slower than 2D. On the machine I ran my experiments on (a 2.1 GHz Xeon, one core), one generation took:

Grid	Cells	Time per generation	Generations per second
64^3	262,144	5.7 ms	175
128^3	2,097,152	51 ms	20
256^3	16,777,216	390 ms	2.5

Compiler settings made a big difference: turning optimization up from -O2 to -O3 made 3D steps 5 to 7 times faster. Memory is never the problem, since each cell is a single bit. On the website, drawing the cubes is the real limit, so it stops at 64^3 .

Random 3D rules I sampled 100 random 3D rules the same way as in 2D and ran each on a 24^3 grid for 200 generations. **All 100 were chaotic.** This makes sense given my 2D finding: 94 of them had a birth count of 4 or less, and with 26 neighbours almost every dead cell has at least 4 live neighbours in a random soup.

To see the rest of the 3D rules, I also sampled rules where each count is only included 20% or 10% of the time (Figure 8). Stable rules came back, and the smallest birth count almost completely decided the category. Every rule with a birth count of 4 or less was chaotic (except one flickering rule), and every rule whose smallest birth count was 9 or more was stable, usually because it died out. The interesting rules sit in between, with smallest birth count 5 to 8, which is also where known 3D Life rules like B5/S45 and B6/S567 are. My classifier found no Complex & Organized 3D rules.

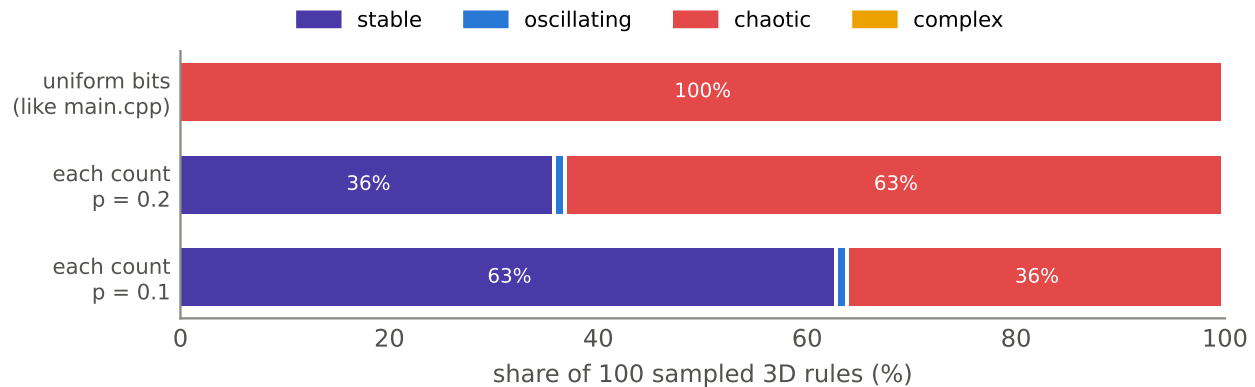


Figure 8: Categories of 100 random 3D rules for three ways of picking rules.

Comparing 2D and 3D

- In both, the smallest birth count decides a lot. In 2D the cutoff is about 3 out of 8 neighbours, in 3D about 5 to 8 out of 26, so in both a cell should need a fair fraction of its neighbours alive to be born.
- Chaos is even more common in 3D (100% vs. 71% of random rules).
- 3D “Life-like” rules are fragile. B5/S45 and B6/S567 usually die down to a few small still lifes within about 100 generations, and I never saw a glider come out of a random soup like they do in Life. The best looking 3D rules are growth rules instead: amoebas, crystals, coral and clouds.

6 Limitations

- Since my grid wraps around, gliders can never escape, so results could differ on an infinite grid.
- The categories depend on how long I run each rule. For example, B45678/S12358 was called complex at 500 generations but settles down by about generation 800.
- The entropy cutoff (0.35) was picked by hand for 2D and reused for 3D. As shown above, flickering rules can fool it, and it can’t see gliders.
- Three starting soups per rule isn’t many, and I only sampled 100 rules per 3D experiment on one grid size.
- The simulation checks every neighbour of every cell one by one. Faster methods (bit tricks or the GPU) would allow much bigger 3D grids.

7 Thoughts

The most surprising thing to me was the HighLife replicator: a 2D rule that ends up drawing a Sierpinski triangle with an exact formula for its population. I was also surprised by how rare “complex” behaviour actually is, and how hard it is to measure. My entropy heuristic sounded reasonable, but the one rule it picked out turned out to be a flickering trick. It was also cool that the smallest birth count acted like a dividing line in both 2D and 3D.

Relating back to the video, the replicator is the closest thing to reproduction I saw, but these rules have no variation or selection, since every run is completely determined by its start. I’d like to learn more about how to automatically detect structures like gliders, and about systems that add mutation, since that seems like the missing piece for something truly life-like.